

RHCSA EX200 — Resumo Dia 08/05

SSH · find · tar · Autofs/NFS · Flatpak · Boot/GRUB · Hostname · SCP

PROVA: 14/05

! Armadilhas do Dia

Certo	Errado
Testar login por chave ANTES de desabilitar PasswordAuthentication	Desabilitar PasswordAuthentication antes de testar — pode travar o acesso (lock-out)
tar -czf (gzip usa -z) · tar -cjf (bzip2 usa -j)	Confundir -z com -j — gera arquivo no formato errado
--exclude='*.pdf' ANTES da fonte, entre aspas simples	--exclude depois da fonte, ou sem aspas — o shell expande o glob antes
-mtime -30 = modificado há MENOS de 30 dias (recente)	Interpretar -30 como 'mais antigo que 30 dias' — o sinal é invertido
fstab NFS sempre com a opção _netdev	Esquecer _netdev — o boot pode travar esperando a rede subir
hostnamectl set-hostname — persiste (grava /etc/hostname)	hostname NOME — muda só em memória, não persiste no reboot
systemctl set-default = permanente (próximos boots)	systemctl isolate = confundir com permanente — isolate só vale para o boot atual

Tabela-Chave — Mapa de Blocos por Simulado

Bloco	Conteúdo	Simulados
1	SSH — chaves, hardening, sshd_config	S1-Q16 · S2-Q19 · S3-Q17 · S4-Q15 · S5-Q19 · S6-Q23
2	find — -size, -mtime, -user, -exec, -perm	S1-Q9 · S2-Q12 · S3-Q14 · S4-Q28 · S5-Q12 · S6-Q14
3	tar — gzip, bzip2, exclusões, -C	S1-Q14 · S2-Q17 · S4-Q14 · S5-Q17 · S5-Q23
4	Autofs / NFS — auto.master + auto.data	S1-Q18 · S2-Q22
5	Flatpak — remote-add + install	S1-Q5 · S2-Q27 · S3-Q9 · S4-Q17 · S5-Q10 · S6-Q15
6	Boot / GRUB / Hostname / /etc/hosts	S1-Q10 · S2-Q10 · S2-Q11 · S4-Q24 · S5-Q20 · S6-Q30
7	Boot Targets — multi-user vs graphical	S1-Q26 · S4-Q7 · S5-Q7 · S6-Q3 · S6-Q7
8	SCP / rsync — transferência segura	S1-Q29

BLOCO 1 — SSH: Chaves e Hardening

A autenticação por chave é mais segura que senha. O fluxo sempre segue a mesma ordem: (1) gerar par de chaves → (2) copiar chave pública → (3) testar → (4) hardening no sshd_config. Nunca desabilite PasswordAuthentication antes de confirmar que o login por chave funciona.

Fluxo completo de referência rápida:

```
# 1. GERAR par de chaves (executar como o usuário que vai usar)
ssh-keygen -t rsa # Enter em todos os prompts (sem passphrase no exame)
# Gera: ~/.ssh/id_rsa (privada) e ~/.ssh/id_rsa.pub (pública)

# 2. COPIAR chave pública para o servidor (pede senha uma única vez)
ssh-copy-id user@servidor
ssh-copy-id root@localhost # testar localmente

# 3. TESTAR login sem senha
ssh user@servidor # deve entrar direto
```

```
# 4. HARDENING – editar /etc/ssh/sshd_config
PasswordAuthentication no          # desabilitar autenticação por senha
PermitRootLogin no                 # bloquear root via SSH (ou yes, se o exame pedir)

# 5. VERIFICAR antes de reiniciar (evita lock-out)
sshd -T | grep -i passwordauthentication
systemctl restart sshd
```

■ SEMPRE teste o login por chave ANTES de desabilitar PasswordAuthentication. Se travar o acesso no exame, você perde pontos sem poder corrigir.

Exemplo completo (S1-Q16): chave RSA para root + desabilitar senha

```
ssh-keygen -t rsa                  # [Enter] em todos os prompts
ssh-copy-id root@localhost         # digitar a senha do root quando pedir
ssh localhost                     # testar – deve entrar sem senha
exit

vim /etc/ssh/sshd_config
# PasswordAuthentication no
sshd -T | grep -i passwordauthentication # confirmar: ...no
systemctl restart sshd
```

Exemplo completo (S2-Q19): criar usuário alice + chaves + hardening duplo

alice precisa ter senha para que o ssh-copy-id consiga autenticar e instalar a chave pela primeira vez.

```
useradd alice
passwd alice                        # senha temporária

ssh-keygen -t rsa
ssh-copy-id alice@serverb          # ou IP se hostname não resolve

ssh alice@serverb                  # testar ANTES de desabilitar senha

vim /etc/ssh/sshd_config
# PermitRootLogin no
# PasswordAuthentication no
systemctl restart sshd
```

Exemplo completo (S3-Q17): padrão invertido — permitir root via SSH

Este simulado pede o oposto do padrão: habilitar o root login em vez de bloquear.

```
vim /etc/ssh/sshd_config
# PermitRootLogin yes
systemctl restart sshd

ssh-keygen -t rsa
ssh-copy-id root@192.168.1.4        # usar IP é mais confiável
ssh root@192.168.1.4               # deve entrar sem senha
```

■ Usar IP em vez de hostname é mais seguro no exame — evita falhas de resolução DNS/hosts que podem impedir o ssh-copy-id de funcionar.

Variação (S5-Q19 / S6-Q23): gerar as chaves como o próprio usuário (não como root)

```
useradd john
passwd john
su - john                          # trocar para john ANTES de gerar as chaves
ssh-keygen -t rsa
ssh-copy-id john@localhost
exit                                # voltar para root

vim /etc/ssh/sshd_config
# PasswordAuthentication no
systemctl restart sshd
ssh john@localhost                 # sem senha
```

■ Fazer o ssh-keygen como o próprio usuário garante que as chaves fiquem no diretório correto (~john/.ssh/). Se fizer como root, a chave fica em /root/.ssh/ — não funciona para o login de john.

BLOCO 2 — find: Filtros Avançados

O find percorre a árvore de diretórios aplicando filtros. Os filtros são AND por padrão (todos precisam ser verdadeiros). A ação `-exec` executa um comando em cada resultado — `{}` é substituído pelo caminho encontrado, e `\;` termina o `-exec`.

Filtro	Significado	Exemplo
<code>-size +3M</code>	Maior que 3 MiB	<code>find /etc -size +3M</code>
<code>-size -5M</code>	Menor que 5 MiB	<code>find /etc -size -5M</code>
<code>-mtime -30</code>	Modificado nos últimos 30 dias	<code>find /etc -mtime -30</code>
<code>-mtime +30</code>	Modificado há mais de 30 dias	<code>find /etc -mtime +30</code>
<code>-type f / -type d</code>	Arquivo regular / diretório	<code>find /usr -type f</code>
<code>-user sam</code>	Pertence ao usuário sam	<code>find /home -user sam</code>
<code>-name '*.conf'</code>	Nome termina em .conf	<code>find /etc -name '*.conf'</code>
<code>-perm -4000</code>	SUID ativo	<code>find /usr -perm -4000</code>

Exemplo completo (S1-Q9): arquivos em /etc > 3MiB → copiar sem sobrescrever

```
mkdir -p /find/largefiles
find /etc -size +3M -exec cp -n '{}' /find/largefiles/ \;
ls /find/largefiles/
```

■ `cp -n` = no-clobber (não sobrescreve). `cp -u` = só copia se a fonte for mais recente. O exame especifica qual usar — leia o enunciado.

Exemplo completo (S2-Q12 / S6-Q14): arquivos de root → copiar preservando o filtro -user

```
mkdir -p /find/rootfiles
find /usr/bin -type f -user root -exec cp {} /find/rootfiles/ \;
ls /find/rootfiles/ | head
```

Exemplo completo (S5-Q12): múltiplos filtros combinados + chmod nos resultados

`-type f`: só arquivos. `-size +5M`: maiores que 5 MiB. `-mtime -30`: modificados há MENOS de 30 dias (recente). `-name '*.conf'`: aspas simples evitam expansão do shell.

```
mkdir -p /find/recent_configs
find /etc -type f -size +5M -mtime -30 -name '*.conf' \
  -exec cp {} /find/recent_configs/ \;

chmod 600 /find/recent_configs/*
ls -l /find/recent_configs/
```

■ `-mtime -30` = modificado há MENOS de 30 dias (recente). `-mtime +30` = modificado há MAIS de 30 dias (antigo). O sinal é invertido em relação ao que parece intuitivo.

Variações do padrão 'copiar preservando atributos' (S4-Q28):

```
mkdir -p /find/sam_files
# cp -rp: recursivo + preserva permissões/dono/timestamps
find /home -user sam -exec cp -rp {} /find/sam_files/ \;
# OU cp -a (archive – ainda mais completo, preserva SELinux contexts)
find /home -user sam -exec cp -a {} /find/sam_files/ \;
```

BLOCO 3 — tar: Criar, Listar e Extrair

O tar empacota e (opcionalmente) comprime arquivos. A flag `-f` deve vir imediatamente antes do nome do arquivo. Ordem mais comum: `tar -czf SAIDA.tar.gz FONTE`.

Flag	Significado	Extensão comum
-c	Create (criar)	.tar.gz / .tar.bz2
-x	eXtract (extrair)	.tar.gz / .tar.bz2
-t	lisT (listar sem extrair)	.tar.gz
-f	File (nome do arquivo)	sempre o último antes do nome
-z	gZip compression	.tar.gz ou .tgz
-j	bzip2 compression	.tar.bz2 ou .tbz2
-J	xz compression	.tar.xz
-C	Change dir (destino)	tar -xzf arq.tar.gz -C /dir
--exclude	Excluir padrão	--exclude='*.pdf'

■ Mnemônico: -z = gZip, -j = bzip2 (bzip2 é 'mais avançado' que gzip), -J = xZ (maiúsculo).

Exemplo completo (S1-Q14): gzip + listar sem extrair

```
tar -czf /root/config_backup.tar.gz /etc/ssh
tar -tf /root/config_backup.tar.gz      # listar SEM extrair
```

Exemplo completo (S2-Q17): bzip2 + extrair para diretório específico

```
mkdir -p /archive/restored/myetc
tar -cjf /archive/myetc.tbz2 /etc      # -j = bzip2
tar -xjf /archive/myetc.tbz2 -C /restored/myetc/  # -C muda o destino
ls /restored/myetc/
```

Exemplo completo (S4-Q14): múltiplas fontes + restaurar

```
mkdir -p /archive/restored
tar -czf /archive/sysconfig.tar.gz /etc /root/anaconda-ks.cfg
tar -tf /archive/sysconfig.tar.gz | head -5

tar -xzf /archive/sysconfig.tar.gz -C /restored
```

Exemplo completo (S5-Q17): gzip excluindo um padrão de arquivo

```
mkdir -p /Backup
tar -czf /Backup/myhome.tgz --exclude='*.pdf' /home
tar -tf /Backup/myhome.tgz | grep '\.pdf'  # não deve retornar nada
```

■ --exclude vem ANTES do caminho de origem. As aspas simples em '*.pdf' são obrigatórias — sem elas, o shell expande o glob antes de passar para o tar.

BLOCO 4 — Autofs / NFS: Montagem sob Demanda

O Autofs monta o compartilhamento NFS apenas quando é acessado e desmonta automaticamente quando inativo — mais eficiente que /etc/fstab para NFS. Requer dois arquivos: auto.master (diretório base + mapa) e um arquivo de mapa.

■ Diferença crítica: fstab NFS = sempre montado (estático). Autofs = monta sob demanda (dinâmico). O exame especifica qual usar.

Exemplo completo (S1-Q18): Autofs para 192.168.1.100:/shares/public

```
dnf install nfs-utils autofs -y

vim /etc/auto.master
# Adicionar: /data /etc/auto.data      (formato: /BASEDIR /ARQUIVO_MAPA)

vim /etc/auto.data
# Conteúdo: public -rw, sync 192.168.1.100:/shares/public
# (formato: SUBDIR OPCOES SERVIDOR:/CAMINHO)

systemctl enable --now autofs
```

```
# TESTAR – o simples acesso dispara a montagem
cd /data/public
ls      # deve mostrar o conteúdo do compartilhamento NFS
```

■ O Autofs cria /data automaticamente — NÃO precisa fazer mkdir. cd /data/public é o que dispara a montagem; sem acesso, não há montagem (economia de recursos).

Exemplo completo (S2-Q22): NFS via fstab (não autofs) com _netdev

```
mkdir /nfs_share          # com fstab o diretório precisa existir antes

vim /etc/fstab
# Adicionar: servera:/share /nfs_share nfs defaults,_netdev 0 0

mount -a                  # montar sem reboot
df -h /nfs_share          # confirmar
```

■ _netdev é OBRIGATÓRIO para NFS no fstab. Sem ele, o sistema pode travar no boot tentando montar o NFS antes da rede estar disponível.

BLOCO 5 — Flatpak: Repositório e Instalação

O Flatpak é o novo padrão RHEL 10 para apps desktop, isolando a aplicação em sandbox. O comando remote-add é equivalente ao arquivo .repo do DNF. --if-not-exists evita erro se o repositório já foi adicionado antes.

Gabarito padrão — todos os simulados seguem a mesma estrutura de 3 comandos:

```
# PASSO 1: adicionar Flathub (uma vez só)
flatpak remote-add --if-not-exists flathub \
  https://dl.flathub.org/repo/flathub.flatpakrepo

# PASSO 2: instalar o app pedido (trocar o ID conforme a questão)
flatpak install flathub [ID_DO_APP] -y

# PASSO 3: verificar
flatpak list
```

Aplicativos já cobrados:

Simulado	Aplicativo	ID Flatpak
S1-Q5	GNOME Text Editor	org.gnome.TextEditor
S2-Q27	GIMP	org.gimp.GIMP
S3-Q9	GNOME Calculator	org.gnome.Calculator
S4-Q17	KCalc (KDE Calculator)	org.kde.kcalc
S5-Q10	Mozilla Firefox	org.mozilla.firefox
S6-Q15	GNOME Calculator	org.gnome.Calculator

■ --if-not-exists é fundamental no exame. Se você já executou o remote-add antes (em outra questão do mesmo simulado), sem essa flag o comando daria erro.

BLOCO 6 — Boot / GRUB / Hostname / /etc/hosts

Exemplo completo (S1-Q10): remover rhgb e quiet do kernel (persistente)

grubby é o método preferido no RHEL — não exige editar o grub.cfg diretamente. --update-kernel=ALL aplica a mudança a todos os kernels instalados.

```
grubby --update-kernel=ALL --remove-args='rhgb quiet'
grubby --info=DEFAULT | grep args      # rhgb e quiet não devem aparecer
```

■ grubby é mais seguro que editar /boot/grub2/grub.cfg diretamente — erros no grub.cfg podem impedir o boot.

Exemplo (S2-Q10): instalar kernel mais recente e confirmar como padrão

```
dnf update kernel -y          # nunca remove o kernel anterior
grubby --default-kernel       # deve mostrar o kernel mais novo
reboot
uname -r                      # confirmar versão nova
```

Exemplo (S2-Q11): hostname estático rhel.server.com

```
hostnamectl set-hostname rhel.server.com
hostnamectl
cat /etc/hostname             # deve mostrar rhel.server.com
```

■ hostnamectl set-hostname atualiza /etc/hostname automaticamente. O comando 'hostname' sozinho só muda em memória — não persiste.

Exemplo completo (S5-Q20): hostname + /etc/hosts para resolução local e externa

```
hostnamectl set-hostname rhel.server.com

vim /etc/hosts
# Garantir estas linhas:
127.0.0.1    localhost localhost.localdomain rhel.server.com
192.168.1.6  rhel.server.com

hostname
ping -c1 rhel.server.com
```

Exemplo (S4-Q24): boot target padrão como rescue.target

```
systemctl set-default rescue.target
systemctl get-default           # rescue.target
```

■ rescue.target exige senha de root para acessar o shell. Depois do exame, lembre de voltar para multi-user.target se necessário.

Exemplo completo (S6-Q30): grubby — listar índices, definir índice 1, remover quiet

Índice 0 = kernel mais novo. Índice 1 = kernel anterior. Definir índice 1 como padrão simula um rollback.

```
grubby --info=ALL | grep -E '^(index|kernel) '

grubby --set-default-index=1
grubby --update-kernel=ALL --remove-args='quiet'
grubby --info=DEFAULT
```

BLOCO 7 — Boot Targets: multi-user vs graphical

Os boot targets do systemd substituem os runlevels do SysV. set-default muda o target permanentemente (próximos reboots). isolate muda o target AGORA, sem reboot.

Target	Equivalente	Quando usar
multi-user.target	Runlevel 3	Servidor sem GUI (economiza RAM/CPU)
graphical.target	Runlevel 5	Desktop com GUI (GNOME instalado)
rescue.target	Runlevel 1	Manutenção (requer senha root)
emergency.target	Mínimo	Recuperação crítica (só root fs)

Exemplo completo (S1-Q26 / S5-Q7 / S6-Q3): multi-user.target

```
systemctl set-default multi-user.target
systemctl get-default           # multi-user.target
```

Exemplo completo (S4-Q7 / S6-Q7): graphical.target

```
systemctl set-default graphical.target
systemctl get-default           # graphical.target
```

■ *set-default: permanente (próximos boots). isolate: temporário (só o boot atual). No exame, use set-default, pois a questão sempre pede persistência.*

BLOCO 8 — SCP / rsync: Transferência Segura

O scp transfere arquivos usando SSH. O rsync é mais eficiente para grandes volumes: só transfere as diferenças e retoma transferências interrompidas. Ambos são aceitos no exame.

Exemplo completo (S1-Q29): copiar arquivo preservando atributos

```
# Opção 1: scp -p (preserva timestamps e permissões)
scp -p /root/anaconda-ks.cfg root@ServerB:/tmp/

# Opção 2: rsync -a (preserva TUDO – dono, permissões, timestamps, links, SELinux)
rsync -av /root/anaconda-ks.cfg root@ServerB:/tmp/

ls -la /tmp/anaconda-ks.cfg
```

Característica	scp	rsync
Preserva atributos	Com -p	Com -a (mais completo)
Transferência parcial	Reinicia do zero	Retoma de onde parou
Sincronia	Sempre copia tudo	Só copia diferenças
Links simbólicos	Não (segue o link)	Sim (com -a)

Tabela Comparativa — Erros Comuns

Situação	Correto ✓	Errado ✗
SSH — ordem de hardening	Testar chave → PasswordAuthentication no	PasswordAuthentication no → testar depois
tar gzip vs bzip2	gzip = -z / bzip2 = -j	tar -czf arq.tbz2 (flag errada)
tar --exclude posição	--exclude='*.pdf' ANTES da fonte	--exclude depois da fonte
find -mtime -30	Menos de 30 dias (recente)	Interpretar como mais antigo
Autofs — diretório base	Autofs cria /data automaticamente	mkdir /data antes (desnecessário)
fstab NFS	Sempre usar _netdev	Sem _netdev — boot pode travar
set-default vs isolate	isolate = agora / set-default = permanente	isolate muda permanentemente
hostname vs hostnamectl	hostnamectl = persiste (grava /etc/hostname)	hostname = só em memória, não persiste

✓ Checklist Final

SSH

- ☐ ssh-keygen -t rsa → ssh-copy-id user@host → ssh user@host (sem senha)
- ☐ PasswordAuthentication no + sshd -T para verificar + systemctl restart sshd
- ☐ PermitRootLogin yes/no conforme o exame pedir — leia o enunciado

find

- ☐ find -size +3M -exec cp -n {} /dest/ \; (copiar sem sobrescrever)
- ☐ find -type f -user root -exec cp {} /dest/ \;

- ☐ `find -size +5M -mtime -30 -name '*.conf'` (múltiplos filtros — todos com AND)
- ☐ `-mtime -30` = RECENTE (menos de 30 dias). `-mtime +30` = ANTIGO (mais de 30 dias)

tar

- ☐ `tar -czf arq.tar.gz /fonte` (criar gzip)
- ☐ `tar -cjf arq.tbz2 /fonte` (criar bzip2) — `-j` não é `-z`
- ☐ `tar -tf arq.tar.gz` (listar sem extrair)
- ☐ `tar -xzf arq.tar.gz -C /destino` (extrair para diretório específico)
- ☐ `tar --exclude='*.pdf' /fonte` (aspas simples obrigatórias, antes da fonte)

Autofs / Flatpak

- ☐ `auto.master: /data /etc/auto.data + auto.data: public -rw SERVER:/path`
- ☐ `fstab NFS com _netdev: servera:/share /mnt nfs defaults,_netdev 0 0`
- ☐ `flatpak remote-add --if-not-exists flathub URL + flatpak install flathub APP -y`

Boot / GRUB / Hostname

- ☐ `grubby --update-kernel=ALL --remove-args='rhgb quiet'`
- ☐ `grubby --set-default-index=1` (rollback para kernel anterior)
- ☐ `hostnamectl set-hostname` + verificar com 'hostname' e `cat /etc/hostname`
- ☐ `/etc/hosts`: mapear hostname para 127.0.0.1 E para o IP externo
- ☐ `systemctl set-default multi-user.target / graphical.target + get-default`

SCP / rsync

- ☐ `scp -p arquivo user@host:/dest` (preservar atributos)
- ☐ `rsync -av arquivo user@host:/dest` (archive — mais completo que `scp -p`)